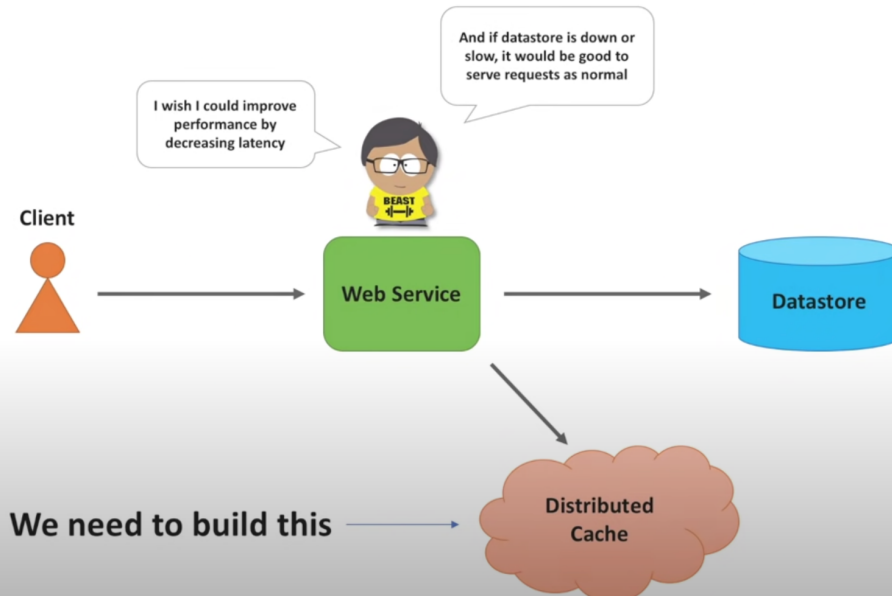


Distributed Cache

 System Design Interview - Distributed Cache

Problem statement



Requirements

Functional

- **put** (key, value)
- **get** (key)

Non-Functional

- **Scalable** (scales out easily together with increasing number of requests and data)
- **Highly Available** (survives hardware/network failures)
- **Highly Performant** (fast puts and fast gets)

One step at a time

Let me start with a local cache. Here is how I would implement in-memory data storage for a single server.

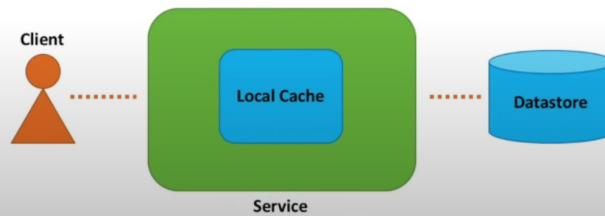


You

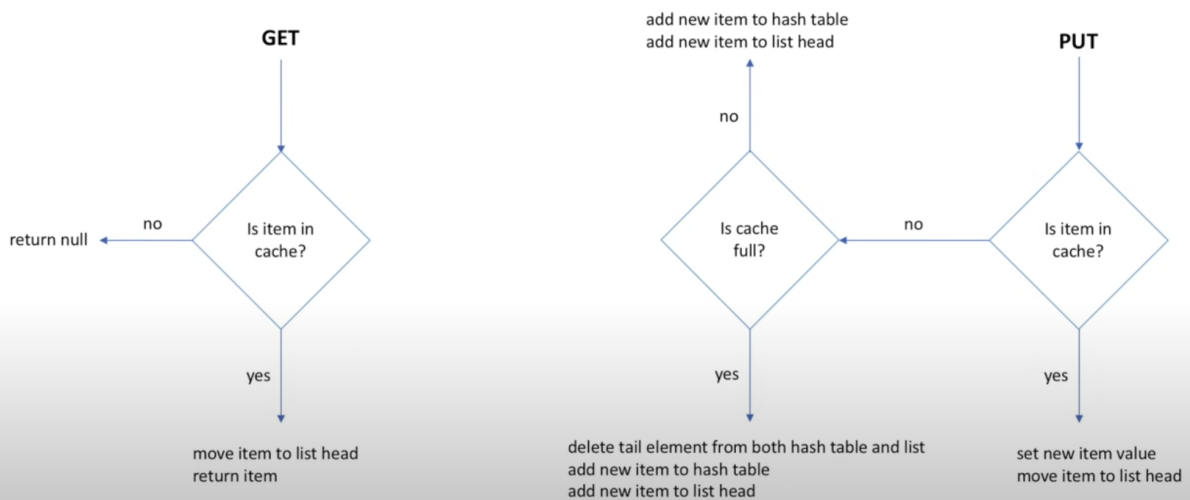
Sounds great! I am specifically interested in data structures you plan to use.



Interviewer



LRU cache algorithm explanation



LRU cache algorithm simulation

Hash table



Doubly linked list



LRU cache algorithm simulation

Hash table

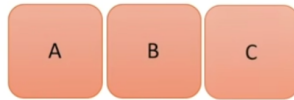


Doubly linked list



LRU cache algorithm simulation

Hash table



Doubly linked list

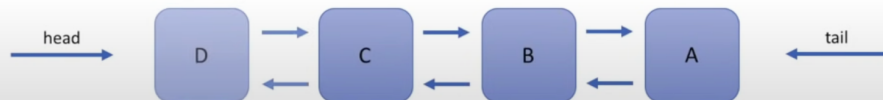


LRU cache algorithm simulation

Hash table

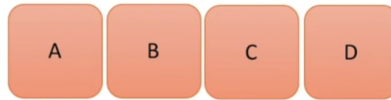


Doubly linked list

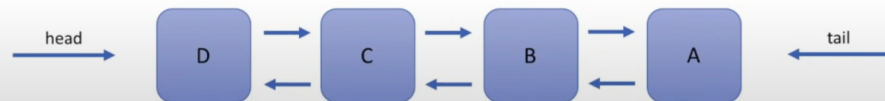


LRU cache algorithm simulation

Hash table



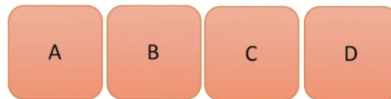
Doubly linked list



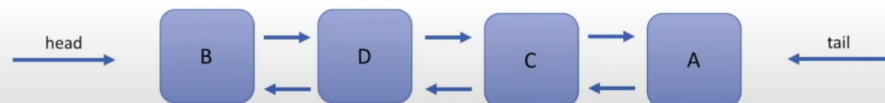
Now, GET operation has been called for item B, and we need to move it to the head.

LRU cache algorithm simulation

Hash table



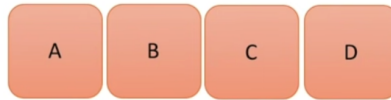
Doubly linked list



Right after that item A has been accessed (also a GET call) and we move it to the head

LRU cache algorithm simulation

Hash table



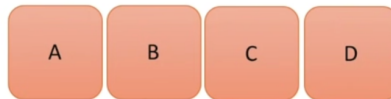
Doubly linked list



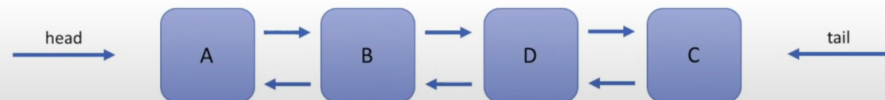
And now, PUT operation has been called.

LRU cache algorithm simulation

Hash table



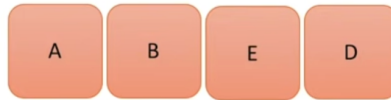
Doubly linked list



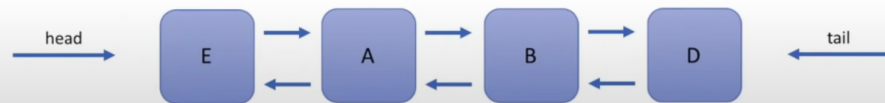
We first delete the item that is at the list tail, because cache is full.

LRU cache algorithm simulation

Hash table



Doubly linked list



And then add item E to both the hash table and the list.

LRU cache algorithm implementation

```
class Node {
    private final String key;
    private String value;

    private Node prev;
    private Node next;

    public Node(String key, String value) {
        this.key = key;
        this.value = value;
    }
}

public class LRUCache {
    private final Map<String, Node> map;
    private final int capacity;

    private Node head = null;
    private Node tail = null;

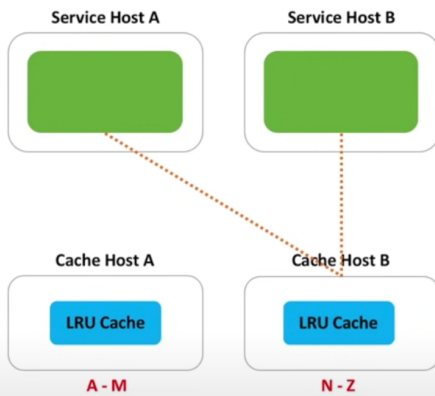
    public LRUCache(int capacity) {
        this.map = new HashMap<String, Node>();
        this.capacity = capacity;
    }

    public String get(String key) {
        if (!map.containsKey(key)) {
            return null;
        }
        Node node = map.get(key);
        deleteFromList(node);
        setListHead(node);
        return node.getValue();
    }

    public void put(String key, String value) {
        if (map.containsKey(key)) {
            Node node = map.get(key);
            node.setValue(value);
            deleteFromList(node);
            setListHead(node);
        } else {
            if (map.size() >= capacity) {
                map.remove(tail.getKey());
                deleteFromList(tail);
            }
            Node node = new Node(key, value);
            map.put(key, node);
            setListHead(node);
        }
    }
}
```

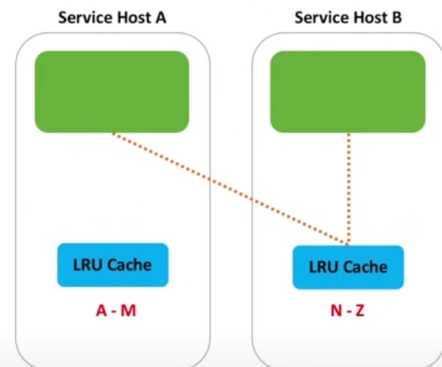
I have omitted implementation for both deleting a node from the list and adding it to the

Stepping into the distributed world



Dedicated cache cluster

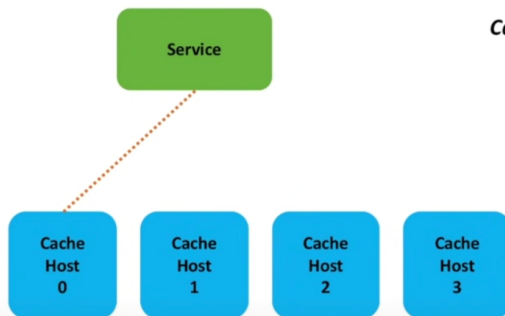
- Isolation of resources between service and cache
- Can be used by multiple services
- Flexibility in choosing hardware



Co-located cache

- No extra hardware and operational cost
- Scales together with the service

Choosing a cache host (naive approach)



$$\text{CacheHostNumber} = \text{HASH_FUNCTION}(\text{key}) \text{ MOD } \text{NumberOfCacheHosts}$$

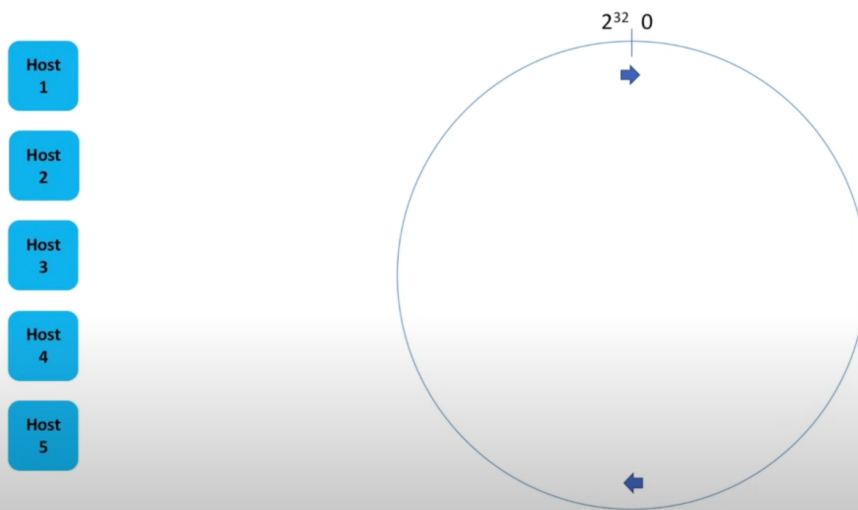
cache key = "some-item-key"

hash = HASH_FUNCTION(cache key), hash = 8

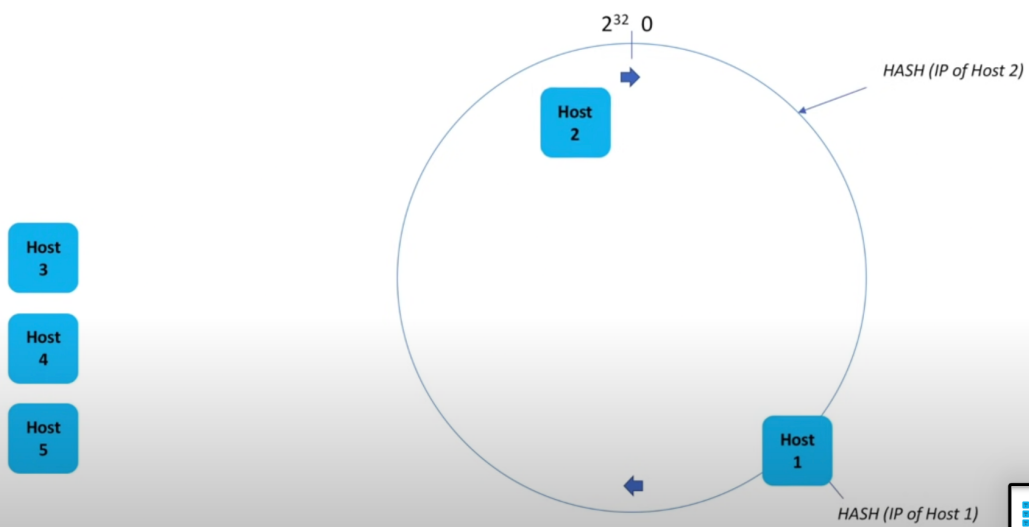
hash MOD 3 = 2

hash MOD 4 = 0

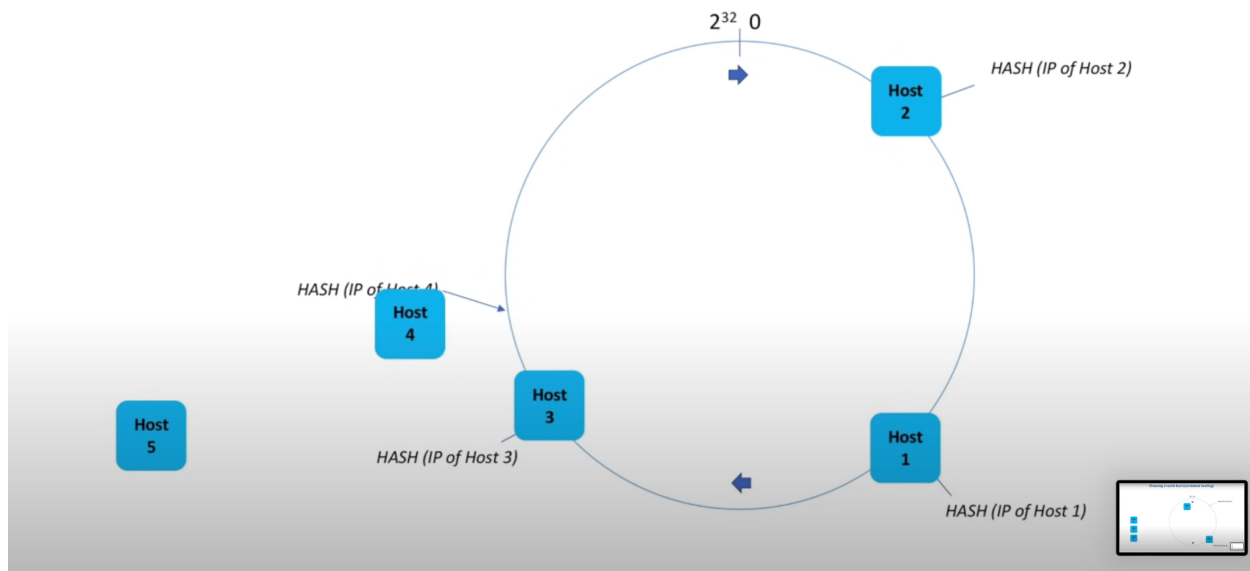
Choosing a cache host (consistent hashing)



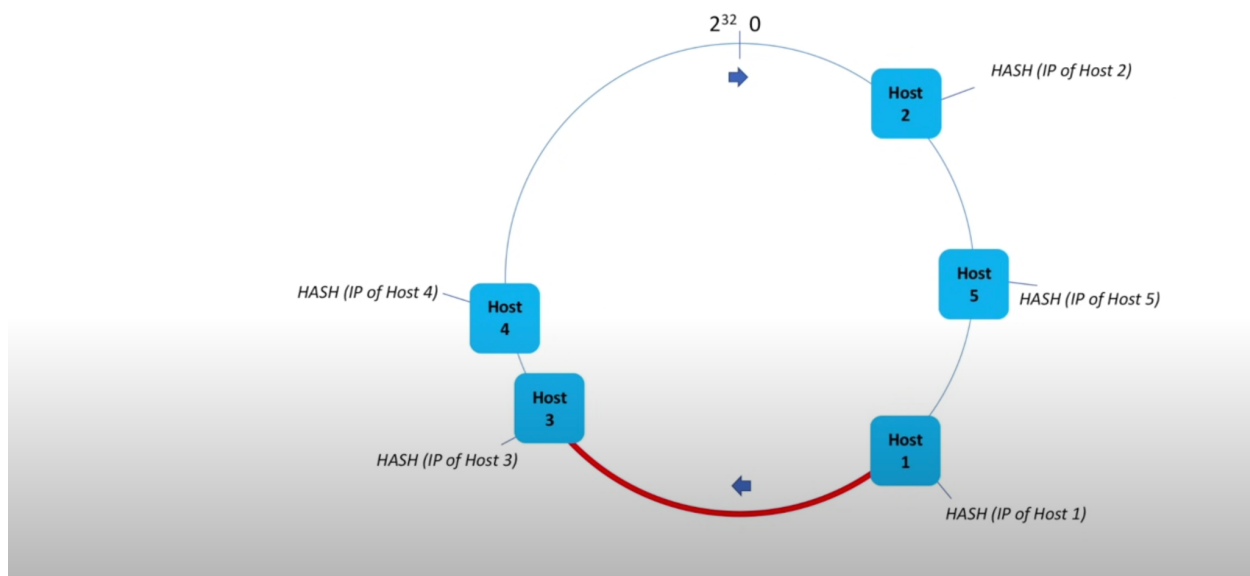
Choosing a cache host (consistent hashing)



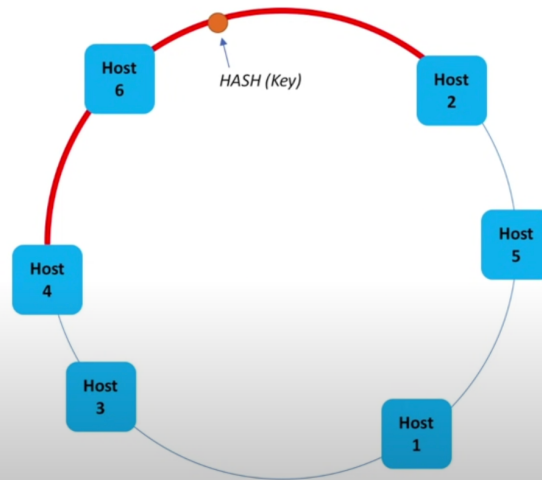
Choosing a cache host (consistent hashing)



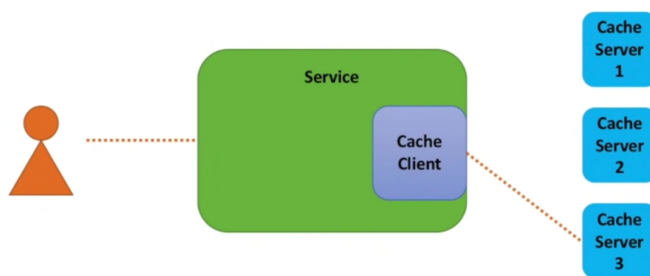
Choosing a cache host (consistent hashing)



Choosing a cache host (consistent hashing)



Cache client

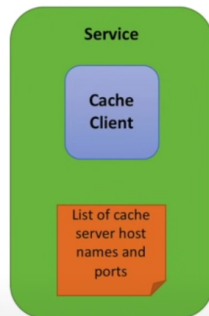


- Cache client knows about all cache servers
- All cache clients should have the same list of servers
- Client stores list of servers in sorted order (by hash value, e.g. TreeMap in Java)
- Binary search is used to identify the server $O(\log n)$
- Cache client uses TCP or UDP protocol to talk to servers
- If server is unavailable, client proceeds as though it was a cache miss

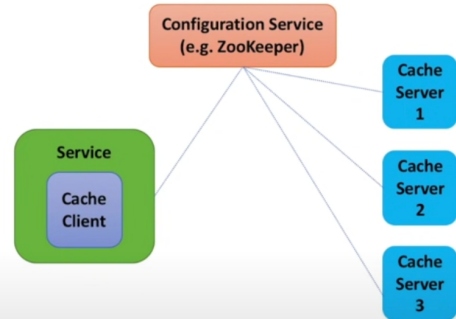
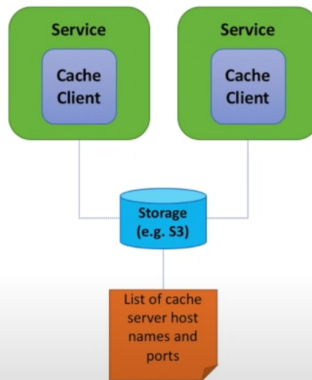


How do clients get the list of servers? How is this list updated when server dies or a new one is added?

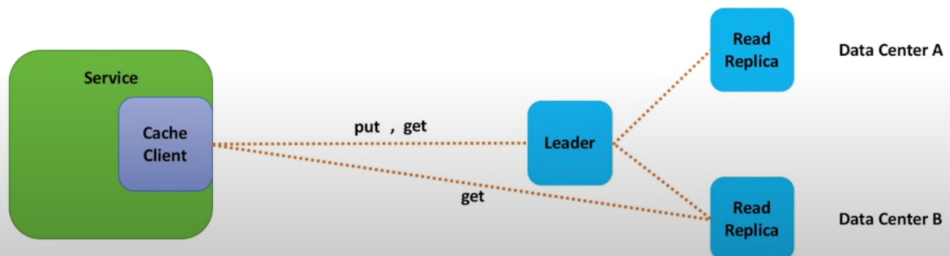
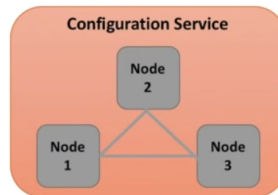
Maintaining a list of cache servers



Use configuration management tools (e.g. Chef, Puppet) to deploy modified file to every service host



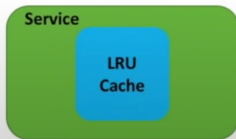
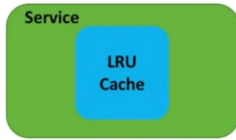
Achieving high availability



What else is important...

- Consistency
- Data expiration
- Local and remote cache
- Security
- Monitoring and logging
- Cache client
- Consistent hashing

Summary



Summary

